

Cypress

Parmis Nouri: 501193499
Silvia Das: 501239223
Abisa Rajkumar: 501250245

Topic Cypress: Public Issue Reporting for Toronto

This is a user-friendly app that helps citizens report issues they notice on the street and track their resolution. To ensure the system is used properly, we need to verify that only real users have access.

The app assists users in identifying the correct organization to contact based on the type of issue. Users additionally have the ability to look at the detailed map, locate the precise area, and report the issue there.

For example, on TMU campus, a student might notice an issue near one of the entrance buildings where they have classes. By reporting it through the app, other students can be informed and find alternative entrances while the problem is being resolved. This way, students can help each other, and issues around campus can be addressed more efficiently.

Campus security also has access to the app, allowing them to track reports and update the status of each problem, ensuring a faster response and resolution.

USER STORIES

Requirement	Story	Initial Priority	Task	Estimate (1=0.5 days ... 5=8 days)
Report a Street Issue (Citizen)	Select a specific location on a provided map to report an issue that occurred there to be sent to the appropriate department	High	Create an interactive map for users to pinpoint issue locations.	8
			Categorize issues (e.g., potholes, broken lights, sidewalk damage, fire hazard).	
			Design a form for users to describe and submit reports.	
			Save reports in a database and link them to user profiles	
	Search for and find the correct phone number of the department or service organization responsible for a specific type of issue in a specific area, so the citizen can call and report it directly.	High	Create a database to store departments, phone numbers, issue types, and locations	
			Add options in the app to select issue type and enter location	
			Match selected issue and location with the correct department	
			Show the department name and phone number in the app	

Attach Photos to Issue Reports (Citizen)	Attach images or videos to issue reports for better verification	Low	Add an option to upload images or videos in the report form.	2
			Ensure only valid image/video formats can be uploaded.	
			Save uploaded files in cloud storage.	
			Connect uploaded media to the reported issue in the database.	
Track Resolution Progress (Citizen)	Receive updates on reported issues via a notification system to track its resolution	High	Create a system to track issue status (“Pending”, “In Progress”, “Resolved”).	4
			Send notifications and emails for status updates.	
			Let officials update issue status from the admin panel.	
			Save status updates with timestamps in the database.	
View Local Reported Issues (Citizens)	View other ongoing issues that have been reported by fellow citizens in a given area	High	Show reported issues on a map with markers.	3

			Add a search bar to filter by category, date, and location. Let users sort by urgency, newest, or nearest issues. Add a filter to show only unresolved issues.	
Update Issue Status (City Officials)	Change the status of an issue (e.g., "Under Review," "In Progress," "Resolved"), so citizens stay informed.	High	Allow officials to update issue status in the admin panel. Add status options like "Under Review," "In Progress," "Resolved." Save status updates in the database with timestamps. Show the updated status on the issue details page.	3
Delete Fake or Inappropriate Reports (City Officials)	Remove fake or offensive reports, so the platform remains trustworthy.	Med	Add a "Delete Report" button for officials. Allow officials to remove fake or inappropriate reports instantly. Delete the report from the database and issue list.	2
Get Notifications for Nearby Issues (Citizens)	Receive alerts about issues newly reported near the citizen's location.	High	Track the user's location. Notify users via push or email when	3

			an issue is reported nearby.	
Search and Filter Issues (Citizens)	Search and filter reported issues based on type, date, or location, to find relevant reports.	Med	Add a search bar for finding reports.	3
			Allow users to filter by type, date, and location.	
			Let users sort reports by newest, nearest, or most urgent.	
			Show filtered results instantly.	
Detect Duplicate Reports	Automatically detect and flag duplicate issues reported in the same area within a short time window.	Med	Define a location radius (e.g., 50m) to group nearby reports	2
			Compare issue types and timestamps	
			Show a warning if a similar issue exists	
			Mark reports as duplicates in the database	
Prevent False Reports	Prevent spam or fake issue reports by verifying users and monitoring suspicious activity.	High	Require email or phone verification during signup	4
			Limit number of reports per hour per account	
			Log suspicious activities for review	

Test Plan for Cypress App

This test plan outlines the core functionality tests for 'Cypress', ensuring that key features work as expected. These test cases focus on verifying basic app functionality, such as navigation, issue reporting, and viewing reported problems on the map. These tests will help confirm that users can interact with the app smoothly and that essential features operate as intended.

The following test cases describe the conditions, test inputs, and expected outputs for each functionality:

1. **User interface navigation:** Ensures that the user is able to navigate without facing any errors.

Test input: A user opens the app and navigates through the home screen, map, and issue reporting page.

Expected output: The app responds smoothly, and all pages are accessible without errors.
2. **Map display:** Ensures that the user can view all issues on the map.

Test input: A user opens the map and selects a specific issue on the map.

Expected output: Reported issues are displayed on the map and when clicked, a description of the issue is displayed.
3. **Report functionality:** Ensures that the user is able to report issues.

Test input: A user taps on a location to indicate where the issue is.

Expected output: The location is marked, its coordinates are stored, and the user can proceed to the next step of the report.

Test input: A user opens the issue reporting page and selects 'Pothole' from the list of problem types.

Expected output: The selection is recorded, and the user can proceed to the next step of the report.

Test input: A user enters a short description and submits the report.

Expected output: The issue is added to the system, appearing on the map, and a confirmation message is displayed.
4. **Status updates:** Ensures users can see the status of reported issues.

Test input: A user opens the map and selects a specific issue on the map.

Expected output: The issue status (e.g., 'Reported', 'In Progress') is displayed.
5. **Notification system:** Ensures users can subscribe for updates on issues.

Test input: A user selects an existing issue and clicks 'Subscribe'.

Expected output: The user is added to the issue's notification list.

As well, the following test cases describe the conditions, test inputs, and expected outputs for different administrative (campus security) functionality:

6. **Viewing reported issues:** Ensures admin can view all reported issues.

Test input: A campus security officer logs in and opens the map.

Expected output: All reported issues are visible, along with details such as timestamps and user reports.

7. **Resolving an issue:** Ensures admin can mark issues as resolved.

Test input: A security officer selects an issue, updates the status to 'Resolved', and submits the change.

Expected output: The issue status updates, and all subscribed users can see the changes made to the original issue.

8. **Preventing false reports:** Ensures admin can flag, remove, and change the status of false/inaccurate reports as needed.

Test input: A security officer reviews an issue and determines it to be false or inaccurate and removes it.

Expected output: The invalid issue is removed from the system.

The test cases above focus on verifying the core functionality of the Cypress app, ensuring that users can report issues, view reported problems, and receive updates. By breaking down the issue reporting process into multiple steps, we can confirm each component works as intended. As well, the inclusion of admin-specific tests ensures that campus security can efficiently manage reported issues, helping maintain the integrity of the platform.

This document provides an overview of our progress, including the product backlog, team velocity, and planned testing for the upcoming sprint. Our focus has been on designing a simplified yet functional version of the Cypress app. Moving forward, we will use this test plan to guide development and ensure that core features work as expected. By prioritizing usability and system functionality, we aim to create an intuitive and effective tool for reporting and tracking issues on campus.